第一阶段实验框架 README

> 面向博士课题第一阶段的可复现实验与代码开发说明。
> 目标是指导 Codex 搭建一条完整的前端实验链，而不是只实现某一个孤立方法。

---

1. 项目定位

本项目对应整篇课题的**第一阶段**，任务不是最终识别，也不是后续反演与决策，而是完成**前端观测质量建模与异常显著性增强**。面对复杂背景、平台扰动和弱异常耦合条件，代码框架需要把原始双传感器长航迹磁观测，稳定转换为三类面向下游的中间结果：高质量异常残差、窗口级质量评分和初步候选异常片段。后续第二阶段并不直接处理原始长序列，而是建立在这一阶段的输出之上，因此第一阶段的代码必须强调"输入规范、表示合理、结果可评估、输出可复用"。

从实现角度看，第一阶段的核心不是单个网络结构，而是一条闭环实验链：原始双传感器输入经过预处理后，先完成总场/梯度双模构造，再进入统一表示、结构化分离与增强、局部检测与质量评分模块，最后输出候选异常片段，并通过下游增益实验验证这些前端结果是否真的能帮助后续阶段。整套框架必须围绕这条链组织，而不能写成只面向某个方法的黑盒脚本。

---

2. 实验目标

第一阶段的实验目标收敛为三个问题：

1. **输入表达是否合理** — 总场与梯度信息是否应当分开处理、实值拼接处理，还是进入统一表示空间后更有利于弱异常保持。
2. **前端分离是否有效** — 背景、平台扰动和局部异常能否在增强过程中被尽量拆开，而不是简单滤波后把目标也一并抹平。
3. **前端输出是否真正有用** — 增强后的残差、评分和候选片段能否改善后续事件构造的输入质量，而不是只生成更好看的波形图。

因此，本阶段所有代码、实验脚本和输出结果都必须服务于以下三个直接产物：

- 增强后的异常残差序列 $\hat{a}(t)$
- 窗口级质量评分 $Q(t)$
- 候选异常片段集合 $\mathcal{C} = \{c_1, c_2, \ldots, c_M\}$

---

3. 输入、输出与整条实验链

3.1 总体处理链路

```
原始双传感器观测 B₁(t), B₂(t)
          |
          ▼
┌──────────────────────────────────┐
| 双模构造与统一表示                |
| Bc(t) = [B₁+B₂]/2                |
| G(t)  = [B₁-B₂]/d(t)             |
```

```
└──────────────────────────────┘
          |
          ▼
┌──────────────────────────────┐
| 结构化分离与增强                 |
| (滑动窗口内完成)                 |
| • 背景项分离                    |
| • 平台扰动项分离                 |
| • 局部异常项分离                 |
└──────────────────────────────┘
          |
          ▼
┌──────────────────────────────┐
| 输出三类中间结果                 |
| ① 增强后异常残差 â(t)          |
| ② 窗口级质量评分 Q(t)          |
| ③ 候选异常片段集合 C            |
└──────────────────────────────┘
```

3.2 输入定义

实验输入是双传感器沿航迹同步采集的磁场观测，记为 $B_1(t), B_2(t)$。若存在平台辅助状态，例如位置、速度、航向、滚转、俯仰、偏航或基线长度，也一并读取，但第一阶段代码不能依赖这些辅助量必须完整存在。

输入数据首先经过时间对齐、统一采样率、异常值处理和滑动窗口切片，然后构造总场/共模通道与梯度/差模通道：

$$B_c(t) = \frac{B_1(t) + B_2(t)}{2}, \qquad G(t) = \frac{B_1(t) - B_2(t)}{d(t)}$$

其中 $d(t)$ 为有效基线长度。

3.3 三种输入表示方案（表示有效性实验的核心变量）

第一阶段必须同时支持**三种输入方案**，作为 Group 1 实验的自变量：

方案	输入形式	说明
方案A	$B_c(t) \in \mathbb{R}^{1\times T}$	单通道总场输入（传统基线）
方案B	$[B_c(t); G(t)] \in \mathbb{R}^{2\times T}$	总场与梯度沿特征维度拼接（实值学习基线）
方案C	$z_t \in \mathbb{R}^D$	双模统一表示（提出方法）

3.4 双模统一表示（方案C）的具体结构

```python
class DualModeEncoder(nn.Module):
    """
    双模统一表示编码器
    目的：在保留总场趋势信息和梯度异常敏感性的同时，学习跨模态协同增强
    """
    def __init__(self, input_dim=2, hidden_dim=64, embed_dim=128):
        # 总场通道 — 捕捉长波长背景趋势
        self.field_encoder = nn.Sequential(
```

```
            nn.Conv1d(1, hidden_dim, kernel_size=7, stride=1, padding=3),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
            ResidualBlock(hidden_dim),
        )
        # 梯度通道 - 捕捉短波长异常
        self.grad_encoder = nn.Sequential(
            nn.Conv1d(1, hidden_dim, kernel_size=5, stride=1, padding=2),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
            ResidualBlock(hidden_dim),
        )
        # 跨模态交叉注意力
        self.cross_attention = CrossModalAttention(hidden_dim)
        # 统一投影
        self.unified_proj = nn.Linear(hidden_dim * 2, embed_dim)

    def forward(self, Bc, G):
        # Bc, G: (batch, 1, T)
        f_feat = self.field_encoder(Bc)      # (batch, hidden_dim, T)
        g_feat = self.grad_encoder(G)          # (batch, hidden_dim, T)

        # 交叉注意力: f→g 和 g→f
        fused = self.cross_attention(f_feat, g_feat)  # (batch, hidden_dim*2, T)

        # 沿时间维度池化 + 投影得到统一表示 z_t
        z = self.unified_proj(fused.mean(dim=-1))      # (batch, embed_dim)
        return z
```

关键约束:

- 方案A/B/C 的后端参数量基本持平（避免"容量更大"干扰）
- 统一表示维度 D 需提前冻结，作为 Group 1 的实验变量
- z_t 的各维度应能与物理分量（背景/扰动/异常强度）建立可解释映射

3.5 检测头与评分头的分工与训练

基本分工

头	角色	优化目标
检测头	回答"这里有没有异常"	高召回，不漏掉真正目标
评分头	回答"这个候选值不值得传下去"	排序稳定性，高质量候选排在前面

联合训练架构

```
class DetectionScoringHead(nn.Module):
    """
    共享早期特征 + 分离检测/评分两个head
    """
    def __init__(self, embed_dim=128):
```

```python
        # 共享特征提取
        self.shared_conv = nn.Sequential(
            nn.Conv1d(embed_dim, 64, kernel_size=3, padding=1),
            nn.ReLU()
        )
        # 检测头：分类 + 定位
        self.det_cls = nn.Conv1d(64, 1, kernel_size=1)   # 有/无异常
        self.det_reg = nn.Conv1d(64, 2, kernel_size=1)   # 峰值位置偏移

        # 评分头：质量评估（独立特征路径）
        self.score_net = nn.Sequential(
            nn.Conv1d(64, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv1d(32, 1, kernel_size=1),            # Q(t) 质量分数
            nn.Sigmoid()
        )

    def forward(self, z_t):
        # z_t: (batch, embed_dim, T)
        feat = self.shared_conv(z_t)            # (batch, 64, T)

        det_cls = self.det_cls(feat)            # (batch, 1, T)  分类
        det_reg = self.det_reg(feat)            # (batch, 2, T)  定位
        q_score = self.score_net(feat)          # (batch, 1, T)  质量分数

        return {
            'det_cls': det_cls,    # D(t)
            'det_reg': det_reg,    # 位置回归
            'q_score': q_score     # Q(t)
        }
```

损失函数设计

```python
class JointLoss(nn.Module):
    def __init__(self, lambda_score=0.3):
        super().__init__()
        self.lambda_score = lambda_score
        self.focal_loss = FocalLoss(alpha=0.25, gamma=2.0)
        self.smooth_l1 = nn.SmoothL1Loss()

    def forward(self, outputs, targets):
        # targets: {labels, reg_target, quality_rank}
        det_cls_loss = self.focal_loss(
            outputs['det_cls'], targets['labels']
        )
        det_reg_loss = self.smooth_l1(
            outputs['det_reg'], targets['reg_target']
        )

        # 评分头: Listwise Ranking Loss
        # 核心思想: 高质量候选（被后续阶段命中的）分数 > 低质量候选
        score_loss = self.listwise_ranking_loss(
```

```
            outputs['q_score'],    # (batch, 1, T)
            targets['keep_label']  # 1=真阳性候选, 0=伪候选
        )

        total_loss = det_cls_loss + det_reg_loss + self.lambda_score * score_loss
        return total_loss

    def listwise_ranking_loss(self, q_score, keep_label):
        """
        同一窗口内: keep_label=1 的 Q(t) 应该高于 keep_label=0
        """
        mask = (keep_label > 0).float()
        pos_score = (q_score * mask).sum() / (mask.sum() + 1e-6)
        neg_score = (q_score * (1 - mask)).sum() / ((1 - mask).sum() + 1e-6)
        return F.relu(1.0 - pos_score + neg_score)  # margin = 1.0
```

评分头标签的定义（Q(t) 权重来源的核心）

评分头的监督信号来自**下游是否命中目标**:

```
def compute_quality_labels(candidates, downstream_hits):
    """
    candidates: 第一阶段输出的候选片段
    downstream_hits: 在仿真/半实物数据上已知目标位置,
                     如果候选片段覆盖了目标 → keep_label=1
    """
    keep_label = torch.zeros_like(candidates.scores)
    for cand in candidates:
        if is_covered_by_target(cand, downstream_hits):
            keep_label[cand.idx] = 1.0
        elif is_false_alarm(cand, downstream_hits):
            keep_label[cand.idx] = 0.0
    return keep_label
```

训练流程（三阶段）

```
阶段1: 先训练检测头（冻结评分头）
  → 用标准异常标签训练 focal loss + smooth L1
  → 检测头收敛后固定

阶段2: 固定检测头, 单独训练评分头
  → 用 keep_label 做 listwise ranking loss
  → 避免两个头相互干扰导致震荡

阶段3: 可选联合微调（低学习率）
  → 两个头一起fine-tune
```

3.6 质量评分 Q(t) 的物理权重设计

评分公式

$$Q(t) = \alpha_1 C_{\text{comp}}(t) + \alpha_2 C_{\text{sig}}(t) + \alpha_3 C_{\text{mode}}(t) + \alpha_4 C_{\text{base}}(t) + \alpha_5 C_{\text{keep}}(t)$$

物理权重初始化

分量	物理含义	初始权重	理由
C_comp	补偿/分离后的净化程度	0.25	背景压得干净是基础
C_sig	局部显著性	0.30	弱异常检测的核心
C_mode	模态一致性	0.20	总场+梯度一致性高 → 真异常概率大
C_base	通道/基线稳定性	0.15	不稳定时段的候选可信度下降
C_keep	任务价值/下游潜力	0.10	间接指标，权重不宜过大

权重敏感性分析

```
WEIGHT_STRATEGIES = {
    'fixed_physical': {
        'alpha': [0.25, 0.30, 0.20, 0.15, 0.10],
        'trainable': False
    },
    'learnable_linear': {
        'alpha': [0.25, 0.30, 0.20, 0.15, 0.10],  # 初始值
        'trainable': True,   # 可学习，但用较大的L2正则约束偏离初始值
        'reg_lambda': 0.01
    },
    'attention_weighted': {
        # 用一个小网络学习输入条件权重
        'use_attention': True
    }
}
```

推荐做法：阶段式权重校准

```
def calibrate_weights(ablation_results):
    """
    对于每个分量 C_i，单独做一次"去掉该分量"的消融
    记录 R_peak 下降幅度 Δ_i
    Δ_i 越大 → 该分量越重要 → 权重正比于 Δ_i
    """
    weight_proportional_to_impact = {
        'comp': abs(delta_peak_when_remove_comp),
        'sig':  abs(delta_peak_when_remove_sig),
        'mode': abs(delta_peak_when_remove_mode),
        'base': abs(delta_peak_when_remove_base),
        'keep': abs(delta_peak_when_remove_keep),
```

```
    }
    return normalize_to_prob(weight_proportional_to_impact)
```

在异常保持实验（Group 3）中，额外增加权重策略对比实验：

- 如果 `fixed_physical` ≈ `learnable` → 说明物理权重合理，不需要学
- 如果 `learnable` 显著更好 → 说明存在数据依赖性，值得微调
- 如果 `learnable` 更差 → 说明过拟合风险高，固定物理权重

3.7 局部检测响应与候选片段构造

局部检测响应记为：

$$D(t) = f_{\text{det}}(\mathbf{z}_t)$$

其中 $\mathbf{z}_t$ 为窗口级特征向量，至少应支持由局部异常强度、窗口能量、峰值持续性、相位一致性、跨分量协同性和局部形态相似性构成。

最终，窗口级结果通过阈值、持续时间和邻接约束聚合为候选异常片段集合。

3.8 第二阶段接口约定（关键）

核心原则

第一阶段输出必须**被当作黑盒外部模块调用**，第二阶段的输入接口必须与实现无关。

数据Schema定义

```
# Stage1Output — 第一阶段的输出格式（冻结的接口合约）
@dataclass
class Stage1Output:
    """第一阶段 → 第二阶段的标准化接口"""

    # 元信息
    trajectory_id: str            # 航迹编号
    total_length: int             # 原始序列长度 T
    sample_rate: float            # 采样率

    # 核心输出
    enhanced_residual: np.ndarray  # â(t), shape=(T,), 增强后异常残差
    quality_score: np.ndarray     # Q(t), shape=(T,), 窗口级质量评分 ∈ [0,1]

    # 候选片段
    candidates: List[Candidate]    # 初步候选异常片段集合

    # 统计信息（用于第二阶段自适应阈值）
    quality_stats: dict            # {mean, std, percentile_90}


@dataclass
class Candidate:
    """候选异常片段"""
    segment_id: int                # 片段编号
```

```python
    start_idx: int                  # 开始索引
    end_idx: int                    # 结束索引
    peak_idx: int                   # 峰值位置
    peak_amplitude: float           # 峰值幅度

    det_confidence: float           # 检测头输出 D(t)，∈ [0,1]
    quality_score: float            # 评分头输出 Q(t)，∈ [0,1]

    # 片段级特征（用于第二阶段事件构造）
    segment_features: np.ndarray    # 固定维度特征向量，e.g., (64,)

    # 可选的中间结果缓存（第二阶段可能用得到）
    residual_clip: np.ndarray       # 该片段对应的增强残差切片


class Stage2EventConstructor(ABC):
    """
    第二阶段事件构造器的抽象接口
    第一阶段输出 → 事件级建模
    """

    @abstractmethod
    def construct_events(self, stage1_output: Stage1Output) -> List[Event]:
        """
        输入: Stage1Output
        输出: Event 列表
        """
        pass

    @abstractmethod
    def score_events(self, events: List[Event]) -> np.ndarray:
        """
        对每个事件打分（用于计算 AUC/F1）
        """
        pass
```

调用约定（无状态接口）

```python
# 第一阶段导出为 .pkl 文件，第二阶段直接加载
# 文件命名规范: {trajectory_id}_stage1.pkl

def load_stage1_output(path: str) -> Stage1Output:
    """第二阶段调用的标准加载函数"""
    with open(path, 'rb') as f:
        return pickle.load(f)
```

下游增益实验的固定后端约定

```
# 实验配置：固定一个最简单的第二阶段构造器
# 目的：隔离第一阶段的贡献，避免第二阶段学习能力掩盖第一阶段的问题
STAGE2_FIXED_BACKEND = SimpleEventConstructor(
    max_gap=5,            # 合并相邻候选的最大间隔
    min_duration=3,       # 最小持续窗口数
    merge_iou_thresh=0.3 # IoU合并阈值
)
```

---

4. 数据三层架构

数据组织必须采用三层结构：仿真数据、半实物数据和实测数据。三类数据不是可选项，而是分别承担"分离正确性验证""近真实背景条件下的异常保持验证"和"真实任务场景鲁棒性验证"三种不同角色。

层级	作用	特点
仿真层	定量检验分离效果	已知异常/背景/扰动，可控真值
半实物层	检验真实背景下的稳定性	真实背景 + 注入偶极子/模板异常
实测层	验证外场鲁棒性与可迁移性	真实复杂场景，无需完美真值

三层数据的验证目标递进关系：

- 仿真层回答"分离对不对"
- 半实物层回答"真实背景里保不保得住"
- 实测层回答"真实任务中能不能用"

仿真数据需要支持扫描目标强度、埋深或等效距离、横向偏距、背景趋势强弱、有色噪声强弱、平台扰动强度和基线误差等因子，并输出背景项、扰动项、目标异常项以及混合观测。

半实物数据以实测背景为底座，在指定时间或空间窗口内注入可控异常，保留注入前后对应关系。

实测数据至少支持三种分组：无目标背景段、可疑异常段和已知目标段。

> ⚠️ **注入异常 vs 真实异常的迁移风险**：半实物数据注入的异常（偶极模型/模板）和真实异常段在特征分布上可能不同。需要在评估指标之外，增加"注入异常 vs 真实异常段特征分布对比"的可视化/统计，作为半实物→实测迁移可信度的额外证据。

三类数据必须共享统一字段定义与统一加载接口，使同一套实验脚本能够通过配置切换数据来源。

---

5. 四组实验设计

实验一：表示有效性（Group 1）

目标：验证输入表达方式本身是否影响弱异常保留和候选输出质量。

方案	输入	后端	方法
方案A	$B_c(t)$ 单通道	SimpleDetector	传统基线
方案B	$[B_c; G]$ 实值拼接	SimpleDetector	实值学习基线
方案C	z_t 统一表示	SimpleDetector	ProposedMethod

输出：典型轨迹波形图、残差统计表、候选片段示意图、可分性可视化。

实验二：结构化分解（Group 2）

目标：验证背景、平台扰动和局部异常是否被有效拆开。

```
# 传统方法：统一用 Bc(t) 输入
# 学习方法：使用与ProposedMethod相同的输入
configs_group2 = {
    'traditional_Bc': {
        'input': 'Bc',
        'methods': ['highpass', 'wavelet', 'EMD', 'VMD', 'OBF', 'gradient_OBF',
'lowrank_sparse']
    },
    'learning_same_input': {
        'input': '[Bc; G]',  # 保持一致，排除输入干扰
        'methods': ['CNN', 'BiLSTM', 'TCN', 'ProposedMethod']
    }
}
```

> ⚠️ 传统方法只使用 Bc(t) 的原因：传统信号处理方法的标准输入是单通道序列，Bc(t) 是最接近"传统输入习惯"的表示；G(t) 本身是 ProposedMethod 提出需要验证的关键输入，如果传统方法加上 G(t) 效果也好，说明"双传感器梯度"的价值是通用的。

实验三：异常保持（最关键，Group 3）

目标：前端在目标逐步变弱、噪声逐步增强、平台扰动逐步加重时，是否仍能保住弱异常。

```
# 核心变量：目标强度逐步降低，固定虚警率，比较召回率曲线
configs_group3 = {
    'input': '[Bc; G]',  # 统一
    'methods': ALL_METHODS,
    'variable': 'target_strength',
    'fixed_far': 0.05
}
```

关键指标：

- 某方法背景净化很好，但峰值衰减严重 → 暴露"净化过度"问题
- **优先级原则：R_peak > ΔSNR** — 弱异常被洗掉无法恢复，背景压不干净可靠后续检测头补救

实验四：下游增益（Group 4）

目标：验证第一阶段是否真的为后续阶段提供了更高质量输入。

```
configs_group4 = {
    'stage1_variants': {
        'traditional_filter': MethodX(output...),
        'real_concat': MethodY(output...),
        'unified_repr': ProposedMethod(output...)
    },
    'stage2_backend': STAGE2_FIXED_BACKEND,  # 固定，不学习
    'compare_metric': 'event_F1'
}
```

这是整条研究链闭环价值的关键：直接回答"第一阶段的改进是否真的能传递给后续阶段"。

---

6. 基线三层体系

基线必须按三层组织，所有基线共享统一数据划分、统一候选构造逻辑和统一评价脚本。

第一层：传统信号处理基线

方法	输入	说明
高通滤波	Bc	经典频率域方法
小波去噪	Bc	时频分析代表
EMD/VMD	Bc	经验模态分解
OBF	Bc	最优带通滤波
梯度OBF	Bc	梯度域最优带通
低秩稀疏分解	Bc	低秩+稀疏分离

第二层：实值学习基线

方法	输入	说明
1D-CNN	[Bc; G]	卷积网络代表
BiLSTM	[Bc; G]	循环网络代表
TCN	[Bc; G]	时序卷积代表
轻量Transformer	[Bc; G]	注意力机制代表

第三层：内部消融基线

消融项	说明
去掉局部耦合信息	验证局部上下文的作用
去掉结构显著性信息	验证结构特征的作用
去掉结构化分离模块	验证分离框架的作用
仅保留简单增强模块	验证增强机制的作用

所有消融实验均使用方案C的统一表示输入，隔离出各模块的独立贡献。

---

7. 关键评价指标

背景净化与增强

- **ΔSNR** = SNR_out - SNR_in：异常相对于背景和噪声的可分辨性提升
- **RMSE_bg**：背景残差均方根误差，衡量背景抑制能力

异常保持能力

- **R_peak** = Â_peak / A_peak（峰值保持率，越接近1越好）
- **Corr_a**：增强后残差与目标真值的相关系数

> ⚠️ **ΔSNR 和 R_peak 的冲突处理原则**：当两者冲突时，**R_peak 优先**。第一阶段的核心约束是"不能漏掉弱异常"，背景压不干净可以靠后续检测头补救，但异常被洗掉就无法恢复。

候选输出能力

- 固定虚警率下的**召回率**
- **候选片段召回率**

下游增益（唯一外部指标）

- 事件级 **F1** 或 **AUC** 提升幅度

---

8. 推荐仓库结构

```
stage1_mag_exp/
├─ README.md
├─ configs/
|  ├─ data/
|  |  ├─ sim.yaml
|  |  ├─ inject.yaml
|  |  └─ real.yaml
|  ├─ model/
|  |  ├─ dual_mode.yaml
```

```
|   |   ├─ detection_scoring.yaml
|   |   └─ stage2_backend.yaml
|   ├─ experiment/
|   |   ├─ repr_effect.yaml
|   |   ├─ decomposition.yaml
|   |   ├─ anomaly_keep.yaml
|   |   └─ downstream_gain.yaml
|   └─ default.yaml
├─ data/
|   ├─ raw/
|   ├─ interim/
|   ├─ processed/
|   └─ metadata/
├─ src/
|   ├─ data/
|   |   ├─ loaders.py          # 统一加载接口
|   |   ├─ preprocess.py        # 时间对齐、采样率统一
|   |   ├─ simulation.py        # 仿真数据生成
|   |   ├─ injection.py         # 半什物数据注入
|   |   └─ field_definition.py  # 统一字段Schema
|   ├─ representation/
|   |   ├─ single_channel.py   # 方案A
|   |   ├─ real_concat.py       # 方案B
|   |   ├─ dual_mode_encoder.py # 方案C
|   |   └─ unified_repr.py      # 统一接口包装
|   ├─ methods/
|   |   ├─ filters.py           # 传统滤波基线
|   |   ├─ wavelet.py            # 小波去噪基线
|   |   ├─ emd_vmd.py            # EMD/VMD基线
|   |   ├─ obf_baselines.py      # OBF/梯度OBF基线
|   |   ├─ lowrank_sparse.py     # 低秩稀疏分解基线
|   |   ├─ cnn_lstm_tcn.py       # 实值学习基线
|   |   └─ structured_enhance.py # 提出方法
|   ├─ detect/
|   |   ├─ feature_extract.py    # z_t 特征构建
|   |   ├─ detect_head.py        # 检测头
|   |   ├─ score_head.py         # 评分头
|   |   ├─ joint_loss.py         # 联合损失
|   |   └─ candidate_build.py    # 候选片段构造
|   ├─ quality/
|   |   ├─ quality_score.py      # Q(t) 五分量计算
|   |   ├─ weight_calibration.py # 权重校准
|   |   └─ ranking_loss.py        # Listwise Ranking Loss
|   ├─ stage2_interface/
|   |   ├─ stage1_output.py       # Stage1Output Schema
|   |   ├─ event_constructor.py   # Stage2EventConstructor 接口
|   |   └─ simple_backend.py      # 下游增益实验的固定后端
|   ├─ eval/
|   |   ├─ metrics_signal.py       # ΔSNR, RMSE_bg, R_peak, Corr_a
|   |   ├─ metrics_candidate.py   # 召回率、候选片段召回率
|   |   ├─ metrics_downstream.py  # 事件级F1/AUC
|   |   └─ plots.py                # 可视化
|   ├─ experiments/
```

```
|   |   ├─ exp_repr_effect.py      # Group 1
|   |   ├─ exp_decomposition.py    # Group 2
|   |   ├─ exp_anomaly_keep.py     # Group 3
|   |   └─ exp_downstream_gain.py  # Group 4
|   └─ utils/
|       ├─ io.py
|       ├─ seed.py
|       └─ logger.py
├─ outputs/
|   ├─ figures/
|   ├─ tables/
|   ├─ logs/
|   └─ checkpoints/
└─ tests/
    ├─ test_data_interface.py
    ├─ test_representation.py
    └─ test_end_to_end.py
```

---

9. 快速开始

开发顺序

1. 完成数据加载、字段统一、预处理和可视化
2. 完成总场/梯度双模构造模块，生成三种输入方案
3. 优先实现传统信号处理基线和低秩稀疏分解基线，建立最小可运行闭环
4. 接入提出的结构化分离与增强方法
5. 实现局部特征提取、检测响应、质量评分和候选片段构造模块
6. 完成四组实验脚本与图表导出

命令流

```python
# 1) 预处理与可视化
python -m src.data.preprocess --config configs/default.yaml

# 2) 生成仿真/注入数据
python -m src.data.simulation --config configs/data/sim.yaml
python -m src.data.injection --config configs/data/inject.yaml

# 3) 跑表示有效性实验（Group 1）
python -m src.experiments.exp_repr_effect --config configs/experiment/repr.yaml

# 4) 跑结构化分解实验（Group 2）
python -m src.experiments.exp_decomposition --config configs/experiment/decomp.yaml

# 5) 跑异常保持实验（Group 3）
python -m src.experiments.exp_anomaly_keep --config configs/experiment/keep.yaml

# 6) 跑下游增益实验（Group 4）
```

```
python -m src.experiments.exp_downstream_gain --config configs/experiment/downstream.yaml
```

---

10. 输出物要求

每次实验运行后，代码应自动导出标准化输出：

- 典型轨迹波形图
- 残差对比图
- 候选片段可视化图
- 异常保持曲线图（Group 3 核心输出）
- 指标统计表（ΔSNR, RMSE_bg, R_peak, Corr_a, 召回率, F1/AUC）
- 方法配置文件副本
- 运行日志
- 中间缓存结果（用于后续复用）

输出文件统一落在 `outputs/figures/`、`outputs/tables/`、`outputs/logs/` 和 `outputs/checkpoints/` 下，命名中包含数据源、方法名和实验编号。

---

11. 验收标准

当以下条件全部满足时，可以认为第一阶段实验框架基本搭建完成：

1. ✅ 原始双传感器数据能够稳定加载、预处理并统一表示
2. ✅ 单通道、实值拼接和双模统一表示三类输入可切换
3. ✅ 传统基线、学习基线和所提出方法均可在统一框架下运行
4. ✅ 四组实验能够自动执行并输出表格和图形
5. ✅ 至少能够稳定报告关键指标 ΔSNR、RMSE_bg、R_peak、固定虚警率下召回率和候选片段召回率
6. ✅ 第一阶段输出结果可被简单的第二阶段模块直接调用，完成至少一组下游增益验证

本质：不仅要"能跑"，还要"能比""能解释""能为下一阶段直接服务"。

---

12. 关键设计决策汇总

决策项	选择	理由
双模统一表示结构	交叉注意力 + 统一投影	保留跨模态协同增强能力
检测头/评分头训练	三阶段序贯训练	避免联合训练震荡
评分头监督信号	下游命中与否	可操作、可验证
Q(t) 权重策略	固定物理权重 + 消融校准	可解释性强于纯学习
ΔSNR vs R_peak 优先级	R_peak 优先	弱异常不可丢失

决策项	选择	理由
传统基线输入	统一使用 Bc(t)	传统方法原生单通道习惯
下游后端	固定规则构造器	隔离第一阶段贡献
三层数据关系	按验证目标递进	仿真→半实物→实测

---

13. 已知风险与应对

风险	应对
注入异常与真实异常分布不同导致半什物→实测迁移失败	增加注入 vs 真实异常的特征分布对比作为额外验证
Q(t) 权重过拟合于仿真数据	权重敏感性分析 + 跨数据集验证
第二阶段未就绪导致下游增益实验悬浮	提前冻结 Stage1Output Schema，第二阶段独立开发但接口兼容
传统基线因输入不同而对比不公平	明确传统基线统一用 Bc(t)，学习型方法统一用 [Bc; G] 或 z_t